

AI-ASSISTED CODING ASSIGNMENT 11.2

Shruthi Addagudi | 2303A52133 | Batch 37

Task Description -1 – (Stack Using AI Guidance)

- **Task:** With the help of AI, design and implement a Stack data structure supporting basic stack operations.

Prompt used:

Generate a python to design and implement a Stack data structure supporting basic stack operations.

Output:

```
Assignment11.py > ...
1  class Stack:
2      def __init__(self):
3          self.items = []
4
5      def push(self, item):
6          """Add an item to the top of the stack"""
7          self.items.append(item)
8
9      def pop(self):
10         """Remove and return the top item from the stack"""
11         if not self.is_empty():
12             return self.items.pop()
13         return None
14
15     def peek(self):
16         """Return the top item without removing it"""
17         if not self.is_empty():
18             return self.items[-1]
19         return None
20
21     def is_empty(self):
22         """Check if the stack is empty"""
23         return len(self.items) == 0
24
25     def size(self):
26         """Return the number of items in the stack"""
```

```

Assignment11.py > ...
1  class Stack:
25     def size(self):
27         return len(self.items)
28
29     def __str__(self):
30         """String representation of the stack"""
31         return str(self.items)
32
33
34 # Example usage
35 if __name__ == "__main__":
36     stack = Stack()
37     stack.push(10)
38     stack.push(20)
39     stack.push(30)
40
41     print("Stack:", stack)
42     print("Peek:", stack.peek())
43     print("Pop:", stack.pop())
44     print("Size:", stack.size())
45     print("Is empty:", stack.is_empty())

```

```

PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding> & C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\python.exe vo/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
Stack: [10, 20, 30]
Peek: 30
Pop: 30
vo/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
Stack: [10, 20, 30]
Peek: 30
Pop: 30
Stack: [10, 20, 30]
Peek: 30
Pop: 30
Peek: 30
Pop: 30
Pop: 30
Size: 2
Is empty: False
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>

```

Explanation:

The stack follows **LIFO (Last In First Out)**.

Elements are added using `push()` and removed using `pop()`.

`peek()` returns the top element without removing it.

`is_empty()` checks whether the stack has elements.

Task Description -2 – (Queue Design)

- **Task:** Use AI assistance to create a Queue data structure following FIFO principles

Prompt used:

Create a Python Queue class implementing FIFO with enqueue, dequeue, front, and size methods using list.

Output:

```
Assignment11.py > ...
1  class Queue:
2      def __init__(self):
3          self.items = []
4
5      def enqueue(self, item):
6          """Add an item to the rear of the queue"""
7          self.items.append(item)
8
9      def dequeue(self):
10         """Remove and return the item from the front of the queue"""
11         if not self.is_empty():
12             return self.items.pop(0)
13         return None
14
15     def front(self):
16         """Return the item at the front without removing it"""
17         if not self.is_empty():
18             return self.items[0]
```

```
Assignment11.py > ...
1  class Queue:
21     def size(self):
23         return len(self.items)
24
25     def is_empty(self):
26         """Check if the queue is empty"""
27         return len(self.items) == 0
28
29
30 # Example usage
31 if __name__ == "__main__":
32     q = Queue()
33     q.enqueue(10)
34     q.enqueue(20)
35     q.enqueue(30)
36
37     print(f"Size: {q.size()}") # Output: 3
38     print(f"Front: {q.front()}") # Output: 10
39     print(f"Dequeue: {q.dequeue()}") # Output: 10
40     print(f"Size: {q.size()}") # Output: 2
```

```
vo/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
Size: 3
Front: 10
Dequeue: 10
vo/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
Size: 3
Front: 10
Dequeue: 10
Front: 10
Dequeue: 10
Dequeue: 10
Size: 2
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding> 
```

Explanation

The queue follows FIFO (First In First Out).

enqueue() adds elements at the rear and dequeue() removes from the front.

front() returns the first element and size() gives total elements.

Task Description -3 –(Singly Linked List Construction)

Task: Utilize AI to build a singly linked list supporting insertion and traversal.

Prompt used:

Build a Python singly linked list with node creation, insertion at end, and traversal display

Output;

Assignment11.py > ...

```
1  class Node:
2      def __init__(self, data):
3          self.data = data
4          self.next = None
5
6  class SinglyLinkedList:
7      def __init__(self):
8          self.head = None
9
10     def insert_at_end(self, data):
11         new_node = Node(data)
12
13         if not self.head:
14             self.head = new_node
15             return
16
17         current = self.head
18         while current.next:
19             current = current.next
20         current.next = new_node
21
22     def display(self):
23         current = self.head
24         elements = []
25
26         while current:
```

```

6 class SinglyLinkedList:
7
8     def display(self):
9         current = self.head
10        elements = []
11
12        while current:
13            elements.append(str(current.data))
14            current = current.next
15
16        print(" -> ".join(elements) if elements else "Empty list")
17
18 # Example usage
19 if __name__ == "__main__":
20     ll = SinglyLinkedList()
21
22     ll.insert_at_end(10)
23     ll.insert_at_end(20)
24     ll.insert_at_end(30)
25     ll.insert_at_end(40)

```

```

size: 2
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding> & C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Le
no/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
10 -> 20 -> 30 -> 40
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>

```

Explanation:

Each node contains **data and a pointer** to the next node.
 Insertion is done at the end by traversing from the head.
 Display prints all elements sequentially.

Task Description -4 – (Binary Search Tree Operations)

Task: Implement a Binary Search Tree with AI support focusing on insertion and traversal.

Prompt used:

Implement a Binary Search Tree in Python with insertion and inorder traversal

Output:


```

Assignment11.py > ...
1  class Node:
2      def __init__(self, value):
3          self.value = value
4          self.left = None
5          self.right = None
6
7  class BinarySearchTree:
8      def __init__(self):
9          self.root = None
10
11     def insert(self, value):
12         if self.root is None:
13             self.root = Node(value)
14         else:
15             self._insert_recursive(self.root, value)
16
17     def _insert_recursive(self, node, value):
18         if value < node.value:
19             if node.left is None:
20                 node.left = Node(value)
21             else:
22                 self._insert_recursive(node.left, value)

```

```

Assignment11.py > ...
7  class BinarySearchTree:
17 def _insert_recursive(self, node, value):
23     else:
24         if node.right is None:
25             node.right = Node(value)
26         else:
27             self._insert_recursive(node.right, value)
28
29     def inorder_traversal(self):
30         result = []
31         self._inorder_recursive(self.root, result)
32         return result
33
34     def _inorder_recursive(self, node, result):
35         if node is not None:
36             self._inorder_recursive(node.left, result)
37             result.append(node.value)
38             self._inorder_recursive(node.right, result)
39
40     # Example usage
41     if __name__ == "__main__":
42         bst = BinarySearchTree()

```

```
40 # Example usage
41 if __name__ == "__main__":
42     bst = BinarySearchTree()
43     values = [50, 30, 70, 20, 40, 60, 80]
44
45     for value in values:
46         bst.insert(value)
47
48     print("Inorder traversal:", bst.inorder_traversal())
```

size: 2
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding> & C:\Users\Lenovo\AppData\Local\Programs\Python\python313\python.exe "c:/Users/Le
no/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
10 -> 20 -> 30 -> 40
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding> & C:\Users\Lenovo\AppData\Local\Programs\Python\python313\python.exe "c:/Users/Le
no/OneDrive/Desktop/AI Assisted Coding/Assignment11.py"
Inorder traversal: [20, 30, 40, 50, 60, 70, 80]
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>

Explanation

BST stores elements such that:

Left subtree → smaller values

Right subtree → larger values

Inorder traversal prints elements in **sorted order**.

Task Description -5 – (Hash Table Implementation)

Task: Create a hash table using AI with collision handling

Prompt used:

Output:

Assignment11.py > ...

```
1  class HashTable:
2      def __init__(self, size=10):
3          self.size = size
4          self.table = [[] for _ in range(size)]
5
6      def _hash(self, key):
7          return hash(key) % self.size
8
9      def insert(self, key, value):
10         index = self._hash(key)
11         for i, (k, v) in enumerate(self.table[index]):
12             if k == key:
13                 self.table[index][i] = (key, value)
14             return
15         self.table[index].append((key, value))
16
17     def search(self, key):
18         index = self._hash(key)
19         for k, v in self.table[index]:
20             if k == key:
21                 return v
22         return None
23
24     def delete(self, key):
25         index = self._hash(key)
26         for i, (k, v) in enumerate(self.table[index]):
27             if k == key:
28                 self.table[index].pop(i)
29             return True
```

```

31
32     def display(self):
33         for i, bucket in enumerate(self.table):
34             print(f"Index {i}: {bucket}")
35
36
37 # Example usage
38 if __name__ == "__main__":
39     ht = HashTable(5)
40
41     ht.insert("name", "Alice")
42     ht.insert("age", 25)
43     ht.insert("city", "New York")
44
45     print("Search 'name':", ht.search("name"))
46     print("Search 'age':", ht.search("age"))
47
48     ht.delete("age")
49     print("Search 'age' after delete:", ht.search("age"))
50
51     ht.display()

```

```

PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding> & C:\Users\Lenovo\AppData\Local\Programs\Python\Python313\python.exe vo/OneDrive/Desktop/AI Assisted Coding/Assignment11.py
Search 'name': Alice
Search 'age': 25
Search 'age' after delete: None
Index 0: [('city', 'New York')]
Index 1: []
Index 2: []
Index 3: [('name', 'Alice')]
Index 4: []
PS C:\Users\Lenovo\OneDrive\Desktop\AI Assisted Coding>

```

Explanation

Hash table uses a **hash function** to map keys to indices.
 Collisions are handled using **chaining** (list at each index).
 Supports insert, search, and delete operations

Conclusion:

Using AI assistance, fundamental data structures such as Stack, Queue, Linked List, BST, and Hash Table were successfully implemented. AI helped in generating optimized code, adding documentation, and improving understanding of data structure operations and performance.

